



Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

FACULTAD DE CIENCIAS

ESCUELA DE INFORMÁTICA

INGENIERÍA DE SOFTWARE II (INF-5250 Z05)

Prof. José Ariel Pereyra

Plan de Mejora de Proceso

Sistema de Gestión de Colegio

Grupo

#1

Integrantes:

- Carol Esther Araujo Rodríguez - 100597534
- Nayeli Alexandra Pacheco Lora - 100632915

Índice

Contenido

1. Introducción.....	2
2. Objetivos.....	2
3. Descripción del Proceso de Desarrollo Actual.....	2
4. Problemas Identificados en el Proceso.....	3
4.1 Problemas Técnicos.....	3
4.2 Problemas de Proceso y Organización	4
6. Acciones de Mejora Propuestas	5
6.1 Mejoras al Proceso Técnico.....	5
6.2 Mejoras al Proceso de Organización	6
8. Plan de Implementación de Mejoras	7
9. Indicadores de Éxito del Plan de Mejora	7
10. Conclusiones	8

1. Introducción

El presente Plan de Mejora de Proceso documenta las lecciones aprendidas, los problemas identificados y las acciones de mejora implementadas o propuestas durante el desarrollo del Sistema de Gestión de Colegio. Su propósito es analizar los procesos de desarrollo utilizados por el Grupo #1, identificar áreas de oportunidad y establecer prácticas mejoradas que eleven la calidad del producto final y la eficiencia del equipo.

La mejora continua de procesos es un principio fundamental en la Ingeniería de Software. Este plan aplica el ciclo PDCA (Planificar, Hacer, Verificar, Actuar) para analizar cada área del proceso de desarrollo y proponer mejoras concretas y medibles.

2. Objetivos

- Identificar los problemas y obstáculos que afectaron el proceso de desarrollo durante el proyecto.
- Documentar las lecciones aprendidas por el equipo en cada fase del proyecto.
- Proponer acciones de mejora concretas para cada problema identificado.
- Establecer mejores prácticas que el equipo aplicará en proyectos futuros.
- Contribuir a la madurez del proceso de desarrollo del equipo mediante la reflexión sistemática.

3. Descripción del Proceso de Desarrollo Actual

El equipo adoptó un proceso de desarrollo iterativo e incremental, organizado en cinco fases según el cronograma del Plan de Proyecto. A continuación se describe cómo se ejecutó cada fase:

Fase	Descripción	Semanas	Herramientas Utilizadas
Planificación y Análisis	Levantamiento de requerimientos, definición del stack tecnológico y elaboración de diagramas UML.	1–4	Word, GitHub, Draw.io
Desarrollo: Núcleo del Sistema	Configuración del entorno, autenticación, gestión de usuarios y módulo académico.	4–8	Next.js, Prisma, MySQL, GitHub

Desarrollo: Módulos Secundarios	Módulo financiero, inventario, transporte y comunicaciones.	8–12	Next.js, Prisma, MySQL, GitHub
Pruebas e Integración	Pruebas unitarias, de integración y de usuario (UAT).	12–15	DevTools, MySQL Workbench
Implementación y Cierre	Despliegue, documentación final y presentación.	15–16	GitHub, Next.js

4. Problemas Identificados en el Proceso

4.1 Problemas Técnicos

ID	Problema	Impacto	Fase en que ocurrió
PT-001	Incompatibilidad de Prisma v7 con el schema tradicional. La versión 7 eliminó el soporte de `url` en schema.prisma, lo que generó múltiples errores de migración.	Alto - bloqueó el desarrollo por varias sesiones	Desarrollo (Fase 2 y 3)
PT-002	Pérdida de datos al ejecutar `npx prisma migrate reset`. El comando borra todos los datos de la base de datos sin posibilidad de recuperación.	Alto - se perdieron datos de prueba registrados	Desarrollo (Fase 3)
PT-003	La carpeta `prisma/` fue nombrada con P mayúscula (Prisma) en lugar de minúscula, causando que los comandos fallaran.	Medio - causó errores de migración	Desarrollo (Fase 2)
PT-004	El archivo `.env` no existía porque se usaba `.env.local`. Prisma solo lee `.env`, causando el error P1012.	Medio - bloqueó las migraciones	Desarrollo (Fase 2)
PT-005	Referencias a modelos inexistentes en el código (ej: `prisma.pago`, `seccion` en estudiantes) causaron errores 500 en las APIs.	Alto - los módulos no cargaban datos	Desarrollo (Fase 3)
PT-006	Los archivos de los módulos se copiaron en carpetas incorrectas repetidamente, causando confusión y regresiones.	Medio - requirió múltiples correcciones	Desarrollo (Fase 2 y 3)
PT-007	El enum `TipoEmpleado` tenía valores genéricos (MAESTRO, CAJERO) cuando debían ser descriptivos completos.	Bajo - afectó la presentación de los datos	Desarrollo (Fase 3)

4.2 Problemas de Proceso y Organización

ID	Problema	Impacto	Fase en que ocurrió
PP-001	Falta de un ambiente de desarrollo documentado. No había instrucciones claras de configuración inicial, lo que causó confusión al instalar dependencias.	Medio - perdió tiempo de configuración	Fase 1
PP-002	El repositorio de GitHub se eliminó y recreó múltiples veces, perdiendo el historial de commits.	Alto - se perdió la trazabilidad del proyecto	Todas las fases
PP-003	Los archivos del proyecto no tenían nombres descriptivos, lo que causó que se copiaran en carpetas incorrectas.	Medio - generó errores difíciles de detectar	Fase 2 y 3
PP-004	No se realizaban respaldos de la base de datos antes de ejecutar migraciones o resets.	Alto - causó pérdida de datos de prueba	Fase 3
PP-005	La comunicación de cambios entre los integrantes no fue siempre oportuna, causando que se trabajara sobre código desactualizado.	Medio - generó conflictos de integración	Fase 2 y 3

5. Lecciones Aprendidas

#	Lección Aprendida	Área
1	Siempre usar Prisma v5 o v6 en proyectos nuevos. Evitar actualizar a versiones mayores sin revisar los cambios de breaking changes.	Técnica
2	Nunca ejecutar `prisma migrate reset` en un ambiente con datos reales. En su lugar, modificar las tablas directamente con ALTER TABLE.	Técnica
3	Mantener siempre los dos archivos: `.env` para Prisma y `.env.local` para Next.js. Son obligatorios y cumplen funciones diferentes.	Técnica
4	Los comandos de Prisma deben ejecutarse siempre desde la carpeta raíz del proyecto, nunca desde dentro de `src/`.	Técnica
5	Los nombres de archivos y carpetas deben ser descriptivos y en minúsculas para evitar problemas en diferentes sistemas operativos.	Organización
6	Hacer un respaldo de la base de datos antes de cualquier migración o cambio de schema.	Proceso
7	El repositorio de GitHub es el activo más valioso del proyecto. Nunca eliminarlo; usar `git push --force` para limpiar si es necesario.	Proceso

8	Revisar siempre la compatibilidad de versiones entre Node.js, Prisma y Next.js antes de instalar dependencias.	Técnica
9	Los mensajes de commit descriptivos facilitan encontrar y revertir cambios cuando algo falla.	Proceso
10	Probar cada módulo inmediatamente después de copiarlo, no esperar a tener todos los archivos para verificar que funciona.	Proceso

6. Acciones de Mejora Propuestas

6.1 Mejoras al Proceso Técnico

ID	Acción de Mejora	Problema que resuelve	Responsable	Estado
AM-001	Documentar la versión exacta de cada dependencia en el README.md del proyecto (Node v24, Prisma v5.22.0, Next.js v16).	PT-001	Carol	✓ Implementado
AM-002	Crear un archivo `database/backup.sql` que se actualiza manualmente antes de cada migración importante.	PT-002, PP-004	Nayeli	Pendiente
AM-003	Agregar una sección de 'Comandos Útiles' en el README.md con los comandos de Prisma y sus contextos correctos.	PT-003, PT-004	Carol	Pendiente
AM-004	Mantener el schema.prisma como fuente única de verdad y sincronizarlo con la base de datos usando `prisma db push` en lugar de migraciones en desarrollo.	PT-005	Nayeli	Pendiente
AM-005	Establecer una convención de nombres para los archivos del proyecto: MODULO-tipo.extensión (ej: EMPLEADOS-route.ts).	PT-006	Todo el equipo	✓ Implementado

6.2 Mejoras al Proceso de Organización

ID	Acción de Mejora	Problema que resuelve	Responsable	Estado
AM-006	Crear una guía de configuración del entorno en el README.md con los pasos exactos para que cualquier integrante pueda configurar el proyecto desde cero.	PP-001	Carol	Pendiente
AM-007	Establecer la regla de nunca eliminar el repositorio de GitHub. Usar ramas o `git push --force` para correcciones.	PP-002	Nayeli	✓ Implementado
AM-008	Realizar un respaldo de la base de datos al inicio de cada sesión de trabajo mediante un script SQL en la carpeta `database/`.	PP-004	Carol	Pendiente
AM-009	Establecer una reunión semanal de 15 minutos para sincronizar el estado del proyecto entre los integrantes.	PP-005	Todo el equipo	Pendiente
AM-010	Usar GitHub Issues para registrar los defectos encontrados y asignarlos a un integrante para su corrección.	PP-003, PP-005	Nayeli	Pendiente

7. Análisis de Causa Raíz de los Principales Problemas

Problema	Causa Raíz	Solución Aplicada	Solución Futura
Incompatibilidad de Prisma v7	Se instaló la versión más reciente sin verificar compatibilidad con el proyecto.	Hacer downgrade a Prisma v5.22.0.	Fijar la versión en package.json y documentarla en el README.
Pérdida de datos por migrate reset	No se conocía el impacto del comando antes de ejecutarlo.	Recrear los datos manualmente.	Documentar el peligro del reset y crear política de respaldos previos.
Archivos en carpetas incorrectas	Los archivos tenían nombres genéricos (page.tsx, route.ts) difíciles de distinguir.	Renombrar archivos con prefijo descriptivo.	Mantener convención de nombres y

			estructura clara documentada.
Eliminación del repositorio	Frustración por errores acumulados y desconocimiento de alternativas.	Recrear el repositorio y hacer push --force.	Capacitar al equipo en comandos Git para resolver problemas sin eliminar el repositorio.
APIs con referencias a modelos inexistentes	El schema de Prisma no estaba sincronizado con el código.	Actualizar el schema y ejecutar prisma generate.	Verificar el schema antes de escribir código que use modelos de Prisma.

8. Plan de Implementación de Mejoras

Acción	Fecha Límite	Responsable	Indicador de Éxito
AM-001: Documentar versiones en README.md	Semana 13	Carol	README actualizado con versiones exactas
AM-002: Script de respaldo de BD	Semana 13	Nayeli	Archivo database/backup.sql en el repo
AM-003: Sección de comandos en README	Semana 13	Nayeli	Sección 'Comandos Útiles' visible en GitHub
AM-006: Guía de configuración del entorno	Semana 14	Carol	Cualquier integrante puede configurar el proyecto con la guía
AM-008: Script de respaldo automático	Semana 14	Carol	Script disponible en la carpeta database/
AM-009: Reunión semanal de sincronización	Semana 13 en adelante	Todo el equipo	Reunión realizada cada semana hasta la presentación
AM-010: Uso de GitHub Issues	Semana 13	Nayeli	Issues creados para los defectos pendientes

9. Indicadores de Éxito del Plan de Mejora

Indicador	Valor Antes de la Mejora	Meta Después de la Mejora
Tiempo promedio para resolver un error de configuración	2–3 horas por sesión	< 30 minutos con la guía documentada

Pérdida de datos por migraciones	2 veces durante el proyecto	0 veces en las semanas restantes
Archivos copiados en carpeta incorrecta	5+ veces durante el proyecto	0 veces con la nueva convención
Commits con mensajes descriptivos	~60% de los commits	100% de los commits
Defectos corregidos en la misma sesión donde se detectaron	~40%	≥ 80%
Tiempo de configuración del entorno desde cero	~2 horas	< 30 minutos con la guía

10. Conclusiones

El desarrollo del Sistema de Gestión de Colegio ha sido un proceso de aprendizaje significativo para el equipo. Los problemas técnicos y de organización encontrados, aunque generaron demoras, proporcionaron lecciones valiosas que han fortalecido las competencias del equipo en el uso de tecnologías modernas como Next.js, Prisma y MySQL.

Las principales áreas de mejora identificadas son la gestión de versiones de dependencias, la protección de datos de la base de datos y la comunicación entre integrantes. Las acciones de mejora propuestas en este plan abordan directamente estas áreas y son factibles de implementar antes de la presentación final del proyecto.

La aplicación de este Plan de Mejora de Proceso, en conjunto con el Plan de Aseguramiento de la Calidad y el Plan de Mediciones, garantiza que el equipo finalizará el proyecto con un producto de alta calidad y con procesos de desarrollo más maduros y eficientes.